

1. Project Description and Objectives

The project is the design and implementation of a custom **3D multibody dynamics engine** with a strong focus on **vehicle and suspension dynamics**, but with a core that is general enough to model a wide variety of mechanical systems (robots, mechanisms, linkages, etc.).

The engine should be:

- **Physically meaningful:** types and APIs must map clearly to physical concepts (bodies, frames, transforms, velocities, constraints, forces).
- **Numerically robust:** stable integration, well-conditioned formulations, and careful treatment of floating-point comparisons and tolerances.
- **Modular and extensible:** it must be easy to add new joints, constraint types, contact models, and more advanced features like flexible bodies and control systems.
- **Well-engineered:** clean C++ design, unit tests, CI, and clear separation between core mechanics and higher-level interfaces (e.g. Python bindings).

The **primary application** is to simulate vehicles, especially the suspension system, with enough fidelity to support:

- **Suspension kinematics:** understanding how wheels, arms, and links move relative to the chassis as joints articulate.
- **Ride and handling analyses:** studying how the vehicle responds to road inputs, steering, braking, etc.
- **Design and optimization:** using the solver inside optimization loops (e.g. genetic algorithms, PSO, surrogate models) to tune geometries and parameters.

The **target technology stack** is:

- Core implementation in **C++ (modern but moderate)**.
- **Eigen** as the linear algebra backend (vectors, matrices, quaternions).
- **Catch2** for unit tests.
- **CMake** as the build system.
- **GitHub Actions** (or similar) for continuous integration.
- Later: a **Python interface** (via bindings) for scripting simulations, parameter sweeps, and integration with optimization workflows.

The **long-term functional objectives** include:

1. 3D Rigid Body Mechanics

- General rigid bodies with full 6-DOF state:
 - Position and orientation in 3D.
 - Linear and angular velocities.
- Consistent geometric framework:
 - Frames and transforms (Transform3 or equivalent).
 - Operations like composition, inversion, and application to points/vectors.

2. Kinematic Structures

- Support for serial chains and tree-like kinematic structures.
- Wide range of joints:
 - Revolute, prismatic, spherical, planar...
 - Compound joints where appropriate (e.g. suspension assemblies).
- Computation of forward kinematics:
 - From joint coordinates to body poses.

3. Dynamics

- Correct representation of mass and inertia (spatial inertia).
- Efficient equations of motion:
 - Recursive formulations for articulated bodies (e.g. articulated-body algorithm, RNEA, etc.).
- External forces:
 - Gravity, springs, dampers, actuators, and user-defined forces.

4. Constraints and Contacts

- Holonomic and non-holonomic constraints (e.g. joints, rolling constraints).
- Contact models:
 - Normal forces.
 - Friction (Coulomb-like, regularized).

- Numerical solvers for constraints:
 - Iterative solvers for constraint impulses or Lagrange multipliers.

5. Time Integration & Simulation Loop

- Stable, configurable integrators:
 - Explicit/semi-implicit schemes.
 - Possibly more advanced integrators for stiff systems.
- Mechanisms for fixed-step and variable-step integration.
- Handling of constraint stabilization and drift.

6. Vehicle-Specific Subsystems

- Suspension models (double wishbone, McPherson, multi-link, etc.).
- Simple tire models at first (lumped stiffness, simple friction), with the option to evolve toward more sophisticated models.
- Integration of steering systems, anti-roll bars, etc.

7. Python Interface & High-Level API

- Python bindings for:
 - Defining models (bodies, joints, constraints).
 - Running simulations.
 - Modifying parameters in loops.
- Simple, user-friendly API for:
 - Setting up common systems (e.g. a basic 4-wheel suspension).
 - Running parameter sweeps and optimizations.

8. Validation, Examples, and Optimization Use

- Benchmark against textbook problems:
 - Pendulum, double pendulum, four-bar linkage, etc.
- Vehicle-related validation:
 - Simple suspension benchmarks (kinematic maps, compliance, etc.).
- Examples that show:

- How to embed the engine in optimization pipelines (PSO, GA, etc.).
- How to compute metrics and objective functions relevant to vehicle design.

9. Advanced & Future Objectives

- Flexible bodies and bushings:
 - Basic flexible elements or modal-reduced components.
 - Linear and nonlinear bushings in joints.
- Control systems:
 - Simple controllers (PID, state-feedback) interacting with the mechanical model.
- Co-simulation or coupling:
 - Interfacing with other tools or models if needed.

All of this must sit on top of a **solid, well-tested geometric and mathematical foundation**, which is why the first phases are dedicated to math types, transforms, and orientations rather than “full dynamics” immediately.

2. Complete Project Roadmap with Detailed Steps

The roadmap is structured in phases. Earlier phases are already partially or fully implemented; later phases are planned with detailed intentions so that the path forward is clear. Each phase is divided into concrete steps, with design and testing expectations.

Phase A – Architecture & Project Skeleton

Goal: Establish a clean, maintainable project structure and high-level architecture.

- **A1. Define scope and main components**
 - Identify the main modules:
 - math (vectors, matrices, quaternions, transforms).
 - geometry or kinematics (frames, transforms, kinematic trees).
 - dynamics (inertia, equations of motion).
 - constraints (joints, contacts, generic constraints).

- integration (time stepping, solver orchestration).
- io / interface (Python bindings, configuration).
- Document responsibilities of each module:
 - For instance, math must not know about bodies or joints; dynamics depends on math and geometry, but not on any Python layer.
- **A2. Create project structure**
 - Define directory layout, e.g.:
 - src/mbd for library sources.
 - include/mbd for public headers.
 - tests/ for unit tests.
 - Choose naming conventions:
 - Namespaces (e.g. mbd).
 - Files (e.g. math.hpp, transform3.hpp, rigid_body.hpp).
- **A3. Document architecture**
 - Write a brief architecture overview (even as comments or a README).
 - Clarify how modules depend on each other:
 - math at the base.
 - geometry/kinematics on top of math.
 - dynamics on top of geometry.
 - constraints across geometry and dynamics.
 - integration orchestrating everything.

Phase B – Build System, Tooling & CI

Goal: Ensure the project is easy to build, test, and maintain across platforms.

- **B1. CMake configuration**
 - Define a main library target (e.g. mbd).
 - Define a test target (e.g. mbd_tests).
 - Make it straightforward to:

- Enable/disable tests, examples, and optional components via CMake options.
 - **B2. Dependency management**
 - Integrate Eigen:
 - Ensure proper include paths.
 - No hard-coded system paths.
 - Integrate Catch2 for testing:
 - Either as submodule, subproject, or via a package.
 - Prepare for future bindings (Python) without tying them in yet.
 - **B3. Warnings, sanitizers, and build types**
 - Set up reasonable defaults:
 - Debug, Release, possibly RelWithDebInfo.
 - Enable compiler warnings:
 - Treat important warnings as errors in CI if desired.
 - **B4. Continuous Integration**
 - Configure GitHub Actions:
 - Configure CMake.
 - Build the project.
 - Run unit tests.
 - Keep CI green:
 - Each new step should maintain or restore passing status.
-

Phase C – Core Math & Geometry

Goal: Build a robust foundation of math types and geometric primitives (especially transforms and orientations), with thorough tests.

C1 – Basic Math Types and Operations

- **C1.1 – Eigen-based aliases and helpers**
 - Define core types:
 - `vec3`, `mat3`, `quat` (aligned with Eigen types).

- Possibly more general types (vecX, matX) if needed later.
- Implement helper functions:
 - Norms, dot product, cross product (or just rely on Eigen's methods wrapped where needed).
 - Conversions:
 - Between rotation matrices and quaternions.
 - Between angle-axis representations and quaternions if useful.
- Testing:
 - Check that type aliases behave as expected (e.g. vec3 is 3D).
 - Verify helper functions:
 - Dot and cross product identities.
 - Conversions between representations (round-trip tests).
- **C1.2 – Rigid transforms and orientation integration**
 - **C1.2.1 – Implement Transform3**
 - Define a type representing a rigid transform between frames, e.g.:
 - Stores quat for rotation and vec3 for translation.
 - Operations:
 - Identity transform.
 - Composition:
 - $T_{\text{world_C}} = T_{\text{world_B}} * T_{\text{B_C}}$.
 - Inverse:
 - Obtain $T_{\text{B_world}}$ such that $T_{\text{B_world}} * T_{\text{world_B}} = \text{Identity}$.
 - Application:
 - Apply transform to points and/or vectors in local frame to obtain world coordinates.
 - Testing:
 - Identity:

- Identity * p == p within tolerance.
 - Composition:
 - Applying $T_{\text{world_A}} * T_{\text{A_B}}$ to a point equals applying $T_{\text{A_B}}$ then $T_{\text{world_A}}$.
 - Inverse:
 - $T.\text{inverse()} * (T * p) == p$ within tolerance.
- **C1.2.2 – Clean integration & small API for Transform3**
 - Decide where Transform3 lives (e.g. math or geometry header).
 - Provide a minimal but powerful interface:
 - Constructors:
 - Identity.
 - From quat + vec3.
 - From rotation matrix + translation if it's consistent with project style.
 - Accessors:
 - rotation() returning quat (or a reference).
 - translation() returning vec3 (or a reference).
 - Utility methods:
 - inverse().
 - compose() or operator *.
 - Functions like transformPoint and possibly transformVector.
 - Ensure clear conventions:
 - Direction of mapping: local → world (or vice versa), documented consistently.
 - Add tests:
 - Constructors and accessors work as expected.
 - Composition and inversion tests cover multiple random cases.
- **C1.3 – Angular velocity and quaternion integration**

- Implement skew(const vec3&):
 - Returns a skew-symmetric matrix $[v]_{\times}$ so that $[v]_{\times} w = v \times w$.
- Implement integrate_quat(const quat& q, const vec3& omega, double dt):
 - Integrates orientation under angular velocity omega for a small time step dt.
 - Keeps the quaternion normalized.
- Testing:
 - For many random v, w, ensure skew(v) * w matches v.cross(w) within tolerance.
 - For a constant angular velocity about a principal axis:
 - Integrate for a small number of steps and compare with analytical solution.
 - Check normalization:
 - Norm of quaternion remains ~ 1 .

C2 – Rigid Bodies & Kinematics

Goal: Represent individual rigid bodies and their placement in space, then build kinematic chains/trees.

• C2.1 – Rigid body state representation

- Define a RigidBodyState or similar:
 - Configuration:
 - Position vec3 and orientation quat or Transform3.
 - Velocity:
 - Linear velocity vec3 (in world frame or body frame, choose and document).
 - Angular velocity vec3 (body frame vs world frame, choose and document).
- Provide operations:
 - Advance state using Transform3 and orientation integration tools.
 - Convert velocities between frames if needed.

- **C2.2 – Rigid body parameters**

- Mass and inertia properties:
 - Mass scalar.
 - Center of mass position relative to a body frame.
 - Inertia tensor relative to a chosen body frame.
- Possibly start shaping a RigidBody class:
 - Holds parameters and current state.
 - Provides helper methods like:
 - `worldTransform()` returning `Transform3`.
 - `applyForceAtPoint(...)` to accumulate forces/torques (even if only used later).

- **C2.3 – Kinematic tree representation**

- Define data structures to represent:
 - A set of bodies.
 - Parent-child relationships.
 - Joint types between them.
- Decide on indexing:
 - Body indices, joint indices, etc.
- Choose a convenient “root” convention (e.g. world frame as root, or base body).

- **C2.4 – Forward kinematics**

- Implement functions that:
 - Given joint coordinates (and body/joint definitions).
 - Compute the world pose (`Transform3`) of each body.
- Testing:
 - Simple chains (e.g. 1-DOF pendulum).
 - Compare with known analytical positions for small examples.

C3 – Dynamics of Multibody Systems

Goal: Compute equations of motion and simulate how bodies move under forces and constraints.

- **C3.1 – Inertia representation and spatial algebra**

- Possibly introduce spatial vector representation (6D motion/force vectors).
- Define spatial inertia objects combining:
 - Mass.
 - Center of mass.
 - Rotational inertia.
- Implement basic operations:
 - Spatial inertia application ($\text{force} = I * \text{acceleration}$, etc.) if using spatial algebra.

- **C3.2 – Equations of motion for a single rigid body**

- Start simple:
 - For an unconstrained, isolated rigid body:
 - Define equations of motion based on Newton–Euler.
- Test:
 - Body under gravity, no other forces:
 - Compare with expected free-fall motion.
 - Body under a constant torque:
 - Compare angular acceleration with theoretical result.

- **C3.3 – Equations of motion for kinematic chains**

- Implement recursive algorithms:
 - Recursive Newton–Euler algorithm (RNEA) for inverse dynamics.
 - Articulated Body Algorithm (ABA) or similar for forward dynamics.
- Handle:
 - Multiple bodies in a tree.
 - Joint actuation (torques/forces).

- Testing:
 - Simple chains (double pendulum, etc.).
 - Compare with known results or reference tools if available.

C4 – Joints, Constraints & Limits

Goal: Introduce and manage constraints that enforce relative motion between bodies.

• **C4.1 – Joint models**

- Implement joint classes:
 - Revolute joint (1 DOF).
 - Prismatic joint (1 DOF).
 - Spherical joint (3 DOF).
 - Fixed joint (0 DOF).
- Each joint:
 - Defines the relation between parent and child frames via Transform3.
 - Provides mapping from joint coordinates to relative transforms.
- Tests:
 - Check that forward kinematics using these joints match expected geometry.

• **C4.2 – Constraint formulation**

- Define constraint equations:
 - Holonomic constraints (position-level).
 - Possibly velocity-level constraints.
- Derive Jacobians of constraints:
 - For use in solvers.
- Implement a small constraint framework:
 - Representing constraint equations and their Jacobians.
- Tests:
 - A constrained system that should have limited motion (e.g., a slider constrained to a line).

- Verify that the solver enforces constraints and that positions/velocities respect them.
- **C4.3 – Joint limits and simple compliance**
 - Implement positional limits:
 - For example, revolute joint angle bounds.
 - Basic compliance:
 - Springs/dampers in joints to represent bushings or soft stops.
 - Test:
 - Joint hitting a limit and being “pushed back” via forces or impulses.

C5 – Contacts & Friction (Vehicle-Oriented)

Goal: Enable bodies (especially wheels and suspension elements) to interact through contact and friction with the ground and other bodies.

- **C5.1 – Contact detection interface**
 - Define an abstraction for contact detection:
 - Even if the broad-phase/narrow-phase is simple initially (e.g. spheres and boxes).
 - Represent contact points:
 - Contact position.
 - Normal vector.
 - Penetration depth or gap.
 - Tangent directions for friction.
- **C5.2 – Normal contact models**
 - Implement simple normal force models:
 - Penalty (spring-damper) model.
 - Optionally, complementarity-based formulations later (more advanced).
 - Tests:
 - Body falling onto a plane with a spring-like contact.
 - Convergence to a resting position with damping.

- **C5.3 – Friction models**

- Implement basic Coulomb-like friction with regularization:
 - Static vs dynamic friction approximations.
- Integrate friction into the constraint solver:
 - E.g., iterative solver enforcing friction cone constraints approximately.
- Tests:
 - Block sliding on an inclined plane.
 - Validate friction thresholds.

- **C5.4 – Simple tire-ground models**

- Implement highly simplified tire models suited as a first step:
 - Lumped stiffness and damping in vertical direction.
 - Simple lateral/longitudinal friction limit based on slip.
- Tests:
 - A single wheel rolling over a bump.
 - Compare qualitative behavior with expectations (no advanced tire physics at this stage).

C6 – Time Integration & Simulation Loop

Goal: Provide robust ways to step simulations in time, coordinating forces, constraints, and dynamics.

- **C6.1 – Integrators**

- Start with simple schemes:
 - Explicit Euler, semi-implicit (symplectic) Euler.
- Possibly add:
 - RK2/RK4 for unconstrained or lightly constrained systems.
- Later, consider more specialized integrators for constrained mechanics.

- **C6.2 – Constraint-aware stepping**

- Incorporate constraints and contacts into the time-stepping scheme:
 - Predict positions and velocities.

- Solve constraint equations (e.g. via impulses or multipliers).
- Correct states to satisfy constraints within tolerances.
- Tests:
 - Compare unconstrained vs constrained versions of the same system.
 - Monitor constraint violation over time.
- **C6.3 – Simulation orchestration**
 - Design a simulation loop abstraction:
 - A Simulator or System object that:
 - Holds the multibody system (bodies, joints, constraints).
 - Advances its state by dt.
 - Provides hooks for user-defined forces and events.
 - Tests:
 - Simulate simple systems from initial conditions and check known trajectories.

C7 – Python Interface & High-Level API

Goal: Make the engine usable from Python for scripting, experiments, and optimization.

- **C7.1 – Core bindings**
 - Expose fundamental entities:
 - Bodies, joints, transforms.
 - System definition and simulation stepping.
 - Provide a minimal but expressive API:
 - `add_body`, `add_joint`, `set_state`, `step(dt)`, etc.
- **C7.2 – Convenience layers**
 - Implement helpers to build common systems:
 - Simple pendulum.
 - Double wishbone or basic suspension.
 - Make these accessible from Python in concise form.

- **C7.3 – Integration with optimization workflows**
 - Provide examples of:
 - Running simulations inside loops that adjust parameters.
 - Collecting metrics (e.g. maximum suspension travel, ride height) as objectives.
 - No dedicated optimization library inside the engine itself; focus on being a *good backend*.

C8 – Validation, Benchmarking & Vehicle Use Cases

Goal: Demonstrate correctness, robustness, and practical applicability.

- **C8.1 – Classical mechanical benchmarks**
 - Implement standard test scenarios:
 - Single pendulum, double pendulum.
 - Four-bar linkage.
 - 3D serial chains.
 - Compare:
 - With analytical solutions where available.
 - Between different integrators.
- **C8.2 – Vehicle & suspension benchmarks**
 - Build example vehicle models:
 - Single-corner suspension (quarter car).
 - Simple full-vehicle suspension with basic tire model.
 - Test scenarios:
 - Bump input.
 - Steady-state cornering with simple lateral force model.
 - Braking/acceleration in straight line.
- **C8.3 – Performance and profiling**
 - Measure computational performance:
 - Per step cost.
 - Scaling with number of bodies/joints.

- Identify bottlenecks and optimize:
 - Data layout, caching, algorithmic improvements.

C9 – Advanced Features & Long-Term Extensions

Goal: Prepare the engine for more sophisticated studies and use cases.

- **C9.1 – Flexible bodies**

- Introduce a minimal flexible body model:
 - E.g. modal-reduced beam elements.
- Integrate with existing multibody framework:
 - Flexible bodies attached to rigid bodies via joints.

- **C9.2 – Bushings and advanced joint compliance**

- Replace simple linear springs/dampers with:
 - Nonlinear or multi-axis bushings.
- Use bushings to model:
 - Suspension bushings, engine mounts, etc.

- **C9.3 – Control systems integration**

- Allow controllers to read states and act on forces/torques:
 - Simple PID controllers as a start.
- Example:
 - Active suspension control on a basic vehicle model.

- **C9.4 – Co-simulation and interoperability**

- Investigate coupling with other tools (optional).
- Design interfaces for:
 - State exchange with external simulators.
 - Runtime plug-in models.

This description and roadmap together define the full scope and step-by-step path of the multibody dynamics engine project, from the current math and geometry foundation all the way to advanced vehicle models, Python integration, and long-term extensions like flexible bodies and control.